DATA STRUCTURES & ALGORITHMS · C++

Searching & Sorting in C++ From Linear Scan to Placement-Ready

Everything for searching and sorting: binary search written without off-by-one bugs, lower and upper bound, searching rotated arrays and matrices, binary search on the answer, every classic sort written and traced by hand, merge sort and quicksort in depth, counting and radix sort, comparators, sorting as a first step, the 12 patterns, and a curated problem list.

51	12	21	38
SECTIONS	PATTERNS	CODE BLOCKS	PROBLEMS

Prepared for **Hari** · on-campus placement preparation
Generated 14 October 2026

Contents

PART 0 — Searching and sorting, from absolute zero

0.1 Linear search and binary search

0.2 Why sorting is worth O(n log n)

0.3 The vocabulary

0.4 How to recognise the topic in a problem statement

PART 1 — Binary search, done right

1.1 The invariant is everything

1.2 Computing mid without overflow

1.3 Template 1 — find an exact value (closed interval)

1.4 Template 2 — the first position where a condition becomes true

1.5 Template 3 — the last position where a condition is true

1.6 lower_bound and upper_bound — the STL

1.7 Off-by-one checklist

PART 2 — Binary search beyond "find x in a sorted array"

2.1 Any monotone predicate

2.2 Peaks

2.3 Rotated sorted arrays

2.4 Two-dimensional search

2.5 Binary search on the answer

2.6 k-th smallest by counting

2.7 Median of two sorted arrays (LC 4)

2.8 Binary search on real numbers

PART 3 — The elementary sorts: bubble, selection, insertion

3.1 Bubble sort

3.2 Selection sort

3.3 Insertion sort

3.4 The three side by side

PART 4 — Merge sort, and counting with it

4.1 The algorithm

4.2 Counting during the merge

PART 5 — Quicksort, partitioning and quickselect

5.1 The idea

5.2 Lomuto partition

5.3 The worst case, and how to avoid it

5.4 Three-way partitioning — the Dutch national flag

5.5 Quickselect — the k-th smallest in O(n) average

PART 6 — Sorting without comparisons

6.1 The Ω(n log n) lower bound

6.2 Counting sort

6.3 Radix sort

6.4 Bucket sort

PART 7 — Sorting in C++: the library and comparators

7.1 The functions

7.2 Writing a comparator

7.3 Strict weak ordering — the comparator rule that crashes programs

7.4 A comparator that decides by concatenation (LC 179)

7.5 Stability, and when it matters

PART 8 — Sorting as the first step

8.1 Intervals

8.2 Two pointers after sorting

8.3 Sort to make a greedy correct

8.4 Sort and keep the original index

PART 9 — The complete pattern taxonomy

How to tell 3 and 7 apart

How to tell 9 and 10 apart

PART 10 — How to approach ANY searching or sorting problem

Step 1 — Read the constraints

Step 2 — Is there something monotone?

Step 3 — Pick the template and state the invariant

Step 4 — For sorting problems: what is the key, and does stability matter?

Step 5 — Check the bounds and the types

Step 6 — Dry-run the edge cases

PART 11 — The bug catalogue

PART 12 — Complexity cheat sheet

PART 13 — The problem list

PART 14 — A three-week plan

Quick reference card

PART 0 — Searching and sorting, from absolute zero

Two questions sit under almost every algorithm you will write:

- **Searching:** *where is x?* — or *is it there at all?*
- **Sorting:** *put these in order* — usually so that a later search, scan or greedy step becomes easy.

They belong together because **order is what makes fast searching possible.** In an unordered array, the only way to find x is to look at every element. In a sorted array, a single comparison with the middle element throws away half of what is left — and doing that again and again finds x in about 20 comparisons among a million elements.

> **The single sentence to remember:** binary search works whenever you can look at one point and decide, for certain, which side of it the answer is on. A sorted array is the most common such situation — but far from the only one.

0.1 Linear search and binary search

```cpp
// linear search: O(n), works on anything
for (int i = 0; i < n; i++) if (a[i] == x) return i;
return -1;

// binary search: O(log n), needs a sorted array
int lo = 0, hi = n - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == x) return mid;
    if (a[mid] < x) lo = mid + 1; else hi = mid - 1;
}
return -1;
```

N	LINEAR SEARCH, WORST	BINARY SEARCH, WORST
1 000	1 000	10
1 000 000	1 000 000	20
10^9	10^9	30

Every step halves the search space, so after k steps n / 2^k elements remain; the search ends when that reaches 1, i.e. after **$\log_2 n$** steps.

0.2 Why sorting is worth O(n log n)

Sorting once costs O(n log n). After that:

- **finding** any value is O(log n);
- **duplicates** sit next to each other — one linear scan finds them;
- **pairs** with a given sum can be found with two pointers in O(n);

- **intervals** sorted by start can be merged in one pass;
- the **k-th smallest** is just `a[k − 1]`;
- many **greedy** algorithms ("always take the earliest-ending meeting") are correct only on sorted input.

So a very common shape of solution is: **sort, then do something linear.** O(n log n) total, and often the intended answer.

0.3 The vocabulary

TERM	MEANING
Search space	The range of indices (or values) where the answer can still be
Invariant	A statement that is true before and after every loop iteration — "the answer is in [lo, hi]"
lower_bound(x)	The first position whose value is **≥ x**
upper_bound(x)	The first position whose value is **> x**
Monotone predicate	A yes/no test that is false, false, …, false, true, true, …, true across the search space
Binary search on the answer	Binary search over possible *answers*, using a feasibility check as the predicate
Stable sort	Equal elements keep their original relative order
In-place sort	Uses O(1) (or O(log n)) extra memory
Comparison sort	Decides order only by comparing pairs — cannot beat O(n log n) in the worst case
Inversion	A pair i < j with a[i] > a[j]; a sorted array has none
Partition	Rearranging around a pivot so smaller values come before it and larger after

0.4 How to recognise the topic in a problem statement

THE PROBLEM SAYS	REACH FOR
"sorted array", "find the position", "insert position"	binary search, `lower_bound`
"first / last occurrence", "count of x in sorted array"	lower and upper bound
"rotated sorted array"	binary search, decide which half is sorted
"peak", "mountain", "bitonic"	binary search on the slope
"minimum speed / capacity / time such that ...", "maximise the minimum", "minimise the maximum"	**binary search on the answer**
"k-th smallest" in a structure too big to list	binary search on the value + counting
"sort", "arrange", "order by ..."	`std::sort` with a comparator
values in a small range (ages, grades, colours)	counting sort
"merge intervals", "meeting rooms", "non-overlapping"	sort by start or end, then scan
"count pairs i < j with a[i] > a[j]" (or a similar cross condition)	merge sort and count during the merge
`n ≤ 10^5` and an $O(n^2)$ brute force	sorting or binary search to reach $O(n \log n)$
answer up to 10^9 or 10^{18} and a cheap check	binary search on the answer

PART 1 — Binary search, done right

Binary search is five lines long and famously easy to get wrong: off-by-one indices, infinite loops, an answer that is one too far. The cure is not memorising templates — it is **knowing exactly what `lo` and `hi` mean** at every step.

1.1 The invariant is everything

Before writing the loop, write down what is true about `lo` and `hi`. Two conventions cover everything:

	CLOSED [LO, HI]	HALF-OPEN [LO, HI)
meaning	the answer, if any, is at an index in lo … hi **inclusive**	the answer is in lo … hi − 1
start	`lo = 0, hi = n − 1`	`lo = 0, hi = n`
loop while	`lo <= hi` (range non-empty)	`lo < hi`
discard mid and left	`lo = mid + 1`	`lo = mid + 1`
discard mid and right	`hi = mid − 1`	`hi = mid`
best for	"find this exact value"	"find the first position where …" (lower_bound)

Mixing them — `hi = n` with `lo <= hi`, or `hi = mid` with `lo <= hi` — is where every infinite loop and out-of-bounds read comes from. Pick one, and make every update preserve its meaning.

1.2 Computing mid without overflow

```
int mid = (lo + hi) / 2;          // lo + hi can exceed INT_MAX when both are near 2^31
int mid = lo + (hi - lo) / 2;     // always safe
```

LC 278 (First Bad Version) has n up to $2^{31} - 1$ precisely to catch the first form. Use the second form always.

1.3 Template 1 — find an exact value (closed interval)

```
int lo = 0, hi = n - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == x) return mid;
    if (a[mid] < x) lo = mid + 1;      // mid is too small; so is everything left of it
    else hi = mid - 1;                  // mid is too big;   so is everything right of it
}
return -1;                              // range became empty
```

Use it only when any matching index will do and "not found" is a valid answer.

1.4 Template 2 — the first position where a condition becomes true

Most binary-search problems are really this: there is a **predicate** P(i) that is false for small i and true for large i, and you want the **first true**.

```
index:      0      1      2      3      4      5      6
P(i):     false  false  false  true   true   true   true
                            ▲
                       the first true = the answer
```

```cpp
int lo = 0, hi = n;                    // half-open: n means "never true"
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (P(mid)) hi = mid;              // mid is true: answer is mid or to its left
    else lo = mid + 1;                 // mid is false: answer is strictly right
}
return lo;                             // lo == hi == first true (or n)
```

Why it terminates: `mid` is always strictly less than `hi`, so `hi = mid` shrinks the range, and `lo = mid + 1` shrinks it too. Why it is correct: the invariant "everything before `lo` is false, everything from `hi` on is true" holds after every step, and the loop ends when they meet.

lower_bound is this template with P(i) = `a[i] >= x`; **upper_bound** with P(i) = `a[i] > x`.

1.5 Template 3 — the last position where a condition is true

Sometimes you want the **last true** of a true-then-false predicate — "the largest r with $r^2 \le x$", "the largest distance that still fits all the cows". Mirror the template, and **round mid up**:

```cpp
int lo = LOW, hi = HIGH;               // the answer is in [lo, hi]
while (lo < hi) {
    int mid = lo + (hi - lo + 1) / 2;  // ROUND UP
    if (P(mid)) lo = mid;              // mid works: answer is mid or to its right
    else hi = mid - 1;
}
return lo;
```

Why round up: with `lo = mid` and the usual round-down mid, when `hi = lo + 1` you get `mid = lo`, `lo = mid` changes nothing, and the loop runs forever. Rounding up makes `mid = hi` in that case, so either branch shrinks the range. **Rule: if a branch does `lo = mid`, round mid up. If a branch does `hi = mid`, round down.**

1.6 lower_bound and upper_bound — the STL

```cpp
#include <algorithm>

auto it  = lower_bound(a.begin(), a.end(), x);   // first element >= x
auto jt  = upper_bound(a.begin(), a.end(), x);   // first element >  x
int  pos = it - a.begin();                        // as an index (may be a.size())

bool found = binary_search(a.begin(), a.end(), x);
int  count = jt - it;                             // occurrences of x
auto [l, r] = equal_range(a.begin(), a.end(), x); // both at once
```

```
a = [1, 2, 2, 2, 5, 7]          x = 2

       1    2    2    2    5    7
            ▲              ▲
      lower_bound     upper_bound          count = 4 - 1 = 3
        (index 1)       (index 4)
```

The questions they answer:

QUESTION	ANSWER
first index of x	`lower_bound`, if it is in range and equals x
last index of x	`upper_bound − 1`, if that equals x
how many x	`upper_bound − lower_bound`
first element > x	`upper_bound`
last element < x	`lower_bound − 1`
last element ≤ x	`upper_bound − 1`
insert x keeping order	insert at `lower_bound`

`std::set` and `std::map` have their own `s.lower_bound(x)` member — use it, not the free function, which is O(n) on tree iterators.

1.7 Off-by-one checklist

Before submitting any binary search, check:

1. Does `hi` start at `n − 1` (closed) or `n` (half-open) — and does the loop condition match?
2. Does every branch **shrink** the range? (`lo = mid` needs round-up.)
3. When the loop ends, can `lo` be `n`? Then guard `a[lo]` before reading it.
4. Is `mid` computed without overflow?
5. Did you test an array of **one** element and an answer at **each end**?

PART 2 — Binary search beyond "find x in a sorted array"

2.1 Any monotone predicate

The array does not have to be sorted. What binary search needs is a **monotone predicate**: once it becomes true, it stays true. Examples:

- First Bad Version (LC 278): "version i is bad" — false, …, false, true, …, true.
- Integer square root (LC 69): "r · r ≤ x" — true, …, true, false, …, false (last true).
- Peak in a mountain (LC 852): "a[i] > a[i + 1]" — false while climbing, true while descending.

Name the predicate before you write the loop, and binary search becomes mechanical.

2.2 Peaks

In **LC 162** the array is not a mountain — it can go up and down many times — but `a[−1] = a[n] = −∞`. Compare `a[mid]` with `a[mid + 1]`: if it is going **up**, a peak must exist to the right (keep climbing; you must eventually come down, at worst at the −∞ boundary). Otherwise there is one at `mid` or to the left. O(log n) for *a* peak, not the highest.

2.3 Rotated sorted arrays

A sorted array rotated at some pivot: `[4,5,6,7,0,1,2]`. At every `mid`, **at least one half is still sorted**, and you can tell which by comparing with an end:

```
  [4, 5, 6, 7, 0, 1, 2]
   lo       mid       hi

  a[lo] = 4 <= a[mid] = 7   ->  [lo, mid] is sorted: 4 5 6 7
  target 0 is NOT in [4, 7]  ->  discard the left half, search right
```

```cpp
if (a[lo] <= a[mid]) {                                  // left half sorted
    if (a[lo] <= target && target < a[mid]) hi = mid - 1;
    else lo = mid + 1;
} else {                                                // right half sorted
    if (a[mid] < target && target <= a[hi]) lo = mid + 1;
    else hi = mid - 1;
}
```

- **Find the minimum (LC 153):** compare `a[mid]` with `a[hi]`. If `a[mid] > a[hi]`, the drop is to the right; otherwise the minimum is `mid` or to its left.
- **With duplicates (LC 81, LC 154):** when `a[lo] == a[mid] == a[hi]` you cannot tell which side is sorted — shrink by one (`lo++`, `hi--` or just `hi--`). The worst case becomes O(n) (all equal but one), and that is unavoidable; say so.

2.4 Two-dimensional search

- **Rows sorted and each row starts after the previous one ends (LC 74):** the matrix *is* one sorted array of R·C elements. Binary search on index k and read `m[k / C][k % C]`. O(log(R·C)).
- **Rows sorted and columns sorted, independently (LC 240):** start at the **top-right** corner. If the value is too big, the whole column below is too big — move left. If too small, the whole row to the left is too small — move down. O(R + C). (Starting at the top-left fails: both moves increase the value.)

2.5 Binary search on the answer

The most important idea in this part, and one of the most frequently tested in placements.

The shape: "find the **minimum** X such that something is possible" (or the maximum X). If X works, every larger X also works (or every smaller one). So "X is feasible" is a monotone predicate over X — binary search **the answer itself**, and for each candidate run a **feasibility check**, usually a greedy O(n) scan.

```
speed:      1    2    3    4    5    6    7   ...  11
finishes?   no   no   no   YES  YES  YES  YES ...  YES       (Koko, piles [3,6,7,11], h = 8)
                         ▲
                  the minimum feasible speed
```

The recipe:

1. **What is the answer?** A speed, a capacity, a day, a distance, a maximum sum.
2. **Bounds:** the smallest and largest values it could possibly take.
3. **Feasibility check:** given a candidate, can it be done? Usually one greedy pass.
4. **Direction:** minimise → first true (Template 2); maximise → last true (Template 3).

PROBLEM	ANSWER	FEASIBILITY CHECK	BOUNDS
LC 875 Koko	eating speed	total hours `Σ ceil(p / k) ≤ h`	1 … max pile
LC 1011 ship packages	capacity	greedy days needed ≤ D	max weight … total
LC 410 split array / **book allocation**	largest piece sum	greedy pieces needed ≤ k	max … total
LC 1482 bouquets	day	bouquets possible by that day ≥ m	min … max bloom day
Aggressive cows (GfG)	minimum gap (maximise)	greedy placement fits all cows	1 … max − min
LC 1802 bounded array	peak value (maximise)	minimal total with this peak ≤ maxSum	1 … maxSum

Minimise the maximum and **maximise the minimum** are the two phrases to listen for. **Book allocation** (allocate books to k students to minimise the maximum pages any student reads) and

painter's partition are LC 410 under other names — two of the most common GfG questions in Indian placements.

Cost: O(n · log(range)). With a range up to 10^9, that is about 30 feasibility checks.

2.6 k-th smallest by counting

When the candidates are implied rather than listed — the k-th smallest pair distance (LC 719), the k-th number in a multiplication table (LC 668), the k-th smallest in a sorted matrix — binary search on the **value** v, and count how many candidates are ≤ v. The answer is the first v whose count reaches k. Counting is usually O(n) with two pointers or a formula, so the whole thing is O(n log(range)) without ever building the candidates.

2.7 Median of two sorted arrays (LC 4)

Split both arrays so the left parts together hold half the elements. If the split takes i elements from the shorter array `a`, it must take `j = half − i` from `b`. The split is correct when

```
a[i−1] <= b[j]   and   b[j−1] <= a[i]          (everything on the left <= everything on the right)
```

If `a[i−1] > b[j]`, i is too large; otherwise too small. Binary search on i over the **shorter** array: O(log(min(m, n))). Use ±∞ for indices off either end. It is the hardest standard binary search, and the boundary handling is the whole difficulty.

2.8 Binary search on real numbers

For a real-valued answer (a square root to 6 decimals, the best ratio), run a fixed number of iterations instead of comparing integers:

```
double lo = 0, hi = 1e9;
for (int it = 0; it < 100; it++) {           // each step halves the range
    double mid = (lo + hi) / 2;
    if (P(mid)) hi = mid; else lo = mid;
}
```

100 iterations shrink any double range below its precision. A `while (hi − lo > 1e-9)` loop can run forever when the range is already at floating-point resolution.

PART 3 — The elementary sorts: bubble, selection, insertion

All three are O(n²). You will rarely use them in code — but written rounds ask you to write them, trace them, count their swaps and comparisons, and say which is stable. And insertion sort is genuinely the best choice for tiny or nearly sorted arrays.

3.1 Bubble sort

Walk the array, swapping neighbours that are out of order. After pass 1 the largest value has "bubbled" to the end; after pass k, the largest k are final. If a pass makes **no** swap, the array is sorted — stop.

```
for (int pass = 0; pass + 1 < n; pass++) {
    bool swapped = false;
    for (int j = 0; j + 1 < n - pass; j++)          // the last `pass` are final
        if (a[j] > a[j + 1]) { swap(a[j], a[j + 1]); swapped = true; }
    if (!swapped) break;                             // early exit
}
```

```
start    [5, 1, 4, 2, 8]
pass 1   [1, 4, 2, 5, 8]
pass 2   [1, 2, 4, 5, 8]
pass 3   [1, 2, 4, 5, 8]   no swap: stop
```

O(n²) worst and average, **O(n) on sorted input** thanks to the early exit. Stable. In place.

3.2 Selection sort

For each position i, find the minimum of the unsorted part and swap it into position i.

```
for (int i = 0; i + 1 < n; i++) {
    int mn = i;
    for (int j = i + 1; j < n; j++) if (a[j] < a[mn]) mn = j;
    swap(a[i], a[mn]);
}
```

```
start    [5, 1, 4, 2, 8]
i = 0    [1, 5, 4, 2, 8]   min of a[0..] -> slot 0
i = 1    [1, 2, 4, 5, 8]   min of a[1..] -> slot 1
i = 2    [1, 2, 4, 5, 8]   min of a[2..] -> slot 2
i = 3    [1, 2, 4, 5, 8]   min of a[3..] -> slot 3
```

Always O(n²) comparisons — even on sorted input. Its one virtue: **at most n − 1 swaps**, useful when writes are expensive. **Not stable**: the long-distance swap can jump an element over its equal twin (`[2a, 2b, 1]` → `[1, 2b, 2a]`).

3.3 Insertion sort

Keep a sorted prefix. Take the next element and shift larger ones right until its slot opens up — the way you sort playing cards in your hand.

```cpp
for (int i = 1; i < n; i++) {
    int key = a[i], j = i - 1;
    while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }   // '>' keeps it stable
    a[j + 1] = key;
}
```

```
start    [5, 1, 4, 2, 8]
i = 1    [1, 5, 4, 2, 8]   insert 1
i = 2    [1, 4, 5, 2, 8]   insert 4
i = 3    [1, 2, 4, 5, 8]   insert 2
i = 4    [1, 2, 4, 5, 8]   insert 8
```

O(n²) worst, **O(n + inversions)** in general — so **O(n) on nearly sorted input**. Stable, in place, and the fastest sort for very small arrays; `std::sort` switches to insertion sort for ranges of about 16 elements or fewer.

3.4 The three side by side

	BUBBLE	SELECTION	INSERTION
Best	O(n) (early exit)	O(n²)	**O(n)**
Average / worst	O(n²)	O(n²)	O(n²)
Swaps / writes, worst	O(n²)	**O(n)**	O(n²)
Stable	yes	**no**	yes
Adaptive (faster on nearly sorted)	yes	no	**yes**
When it is the right choice	teaching	expensive writes	tiny or nearly sorted arrays

Written-round favourites: "how many swaps does bubble sort make?" — exactly the number of **inversions**. "Which O(n²) sort is not stable?" — selection. "Which sort is best for an almost sorted array?" — insertion.

PART 4 — Merge sort, and counting with it

Merge sort is covered as divide-and-conquer in Topic 05 Part 4.6. Here the focus is what it lets you **count**.

4.1 The algorithm

Split in half, sort each half recursively, merge two sorted halves with two pointers.

```cpp
void mergeSort(vector<int>& a, int lo, int hi, vector<int>& tmp) {
    if (lo >= hi) return;
    int mid = lo + (hi - lo) / 2;
    mergeSort(a, lo, mid, tmp);
    mergeSort(a, mid + 1, hi, tmp);
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi) tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= hi)  tmp[k++] = a[j++];
    for (k = lo; k <= hi; k++) a[k] = tmp[k];
}
```

```
 [38, 27, 43, 3, 9, 82, 10]

 split:     [38 27 43 3]          [9 82 10]
            [38 27] [43 3]        [9 82] [10]
            [38][27][43][3]       [9][82][10]
 merge:     [27 38] [3 43]        [9 82] [10]
            [3 27 38 43]          [9 10 82]
            [3 9 10 27 38 43 82]
```

O(n log n) always (log n levels, O(n) merging per level), O(n) extra space, **stable** because of the `<=`. It is the natural sort for **linked lists** (no random access needed, and the merge needs no buffer) and for **external sorting** — data too big for memory, sorted in chunks and merged from disk.

4.2 Counting during the merge

At the moment of the merge, **both halves are sorted**, and every left element sits before every right element in the original array. That makes pairs that cross the middle countable in linear time.

Count inversions (GfG classic): when you take `a[j]` from the right half, every element still waiting in the left half — `mid - i + 1` of them — is greater than it and came before it.

```cpp
if (a[i] <= a[j]) tmp[k++] = a[i++];
else { inversions += mid - i + 1; tmp[k++] = a[j++]; }
```

The same idea, with a different count:

PROBLEM	COUNTED AT THE MERGE
count inversions	`a[i] > a[j]`: add `mid − i + 1`
LC 315 count smaller after self	per element — sort *indices*, credit each left element with the right elements already taken
LC 493 reverse pairs, `a[i] > 2·a[j]`	a separate two-pointer sweep before merging
LC 327 range sums in [lower, upper]	merge-sort the **prefix sums**; for each left prefix, a window of right prefixes

The sentence to say: "*Within each half the pairs are counted recursively; across the halves, both sides are sorted, so a two-pointer pass counts them in O(n). That gives T(n) = 2T(n/2) + O(n) = O(n log n).*"

PART 5 — Quicksort, partitioning and quickselect

5.1 The idea

Pick a **pivot**. **Partition** the array so everything smaller comes before it and everything larger after it — the pivot is now at its final sorted index. Recurse on both sides. Merge sort does its work while **combining**; quicksort does its work while **dividing**, and combining is free.

5.2 Lomuto partition

The simplest to write: the last element is the pivot, and `i` marks the end of the "smaller than pivot" region.

```cpp
int partition(vector<int>& a, int lo, int hi) {
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; j++)
        if (a[j] < pivot) swap(a[i++], a[j]);
    swap(a[i], a[hi]);
    return i;                       // the pivot's final index
}
```

```
pivot = 5    (Lomuto invariant: a[0..i-1] < pivot)

start          [7, 2, 9, 4, 3, 8, 5]   i = 0
j=0: 7 >= 5    [7, 2, 9, 4, 3, 8, 5]   skip
j=1: 2 < 5     [2, 7, 9, 4, 3, 8, 5]   swap, i = 1
j=2: 9 >= 5    [2, 7, 9, 4, 3, 8, 5]   skip
j=3: 4 < 5     [2, 4, 9, 7, 3, 8, 5]   swap, i = 2
j=4: 3 < 5     [2, 4, 3, 7, 9, 8, 5]   swap, i = 3
j=5: 8 >= 5    [2, 4, 3, 7, 9, 8, 5]   skip
place pivot    [2, 4, 3, 5, 9, 8, 7]   pivot 5 is final at index 3
```

Hoare's partition — two pointers moving inward from both ends, swapping out-of-place pairs — does about three times fewer swaps and is the original algorithm, but its boundary rules are trickier. Know that it exists; write Lomuto under pressure.

5.3 The worst case, and how to avoid it

If the pivot is always the smallest or largest element, one side of every partition is empty: n + (n − 1) + … = **O(n²)**. With the last element as pivot, that happens on **already sorted input** — the most common input there is.

- **Random pivot:** swap a random element into the pivot slot first. Expected O(n log n) on every input.
- **Median of three:** the median of first, middle and last.
- **Three-way partition** for many duplicates (5.4).

`std::sort` is **introsort**: quicksort that switches to heap sort if recursion gets too deep, and to insertion sort for tiny ranges — O(n log n) worst case guaranteed.

5.4 Three-way partitioning — the Dutch national flag

With many equal keys, two-way partitions keep re-sorting the equal ones. Split into **three** regions instead: `< pivot`, `== pivot`, `> pivot`.

```
  [ < pivot | == pivot |  unseen   | > pivot ]
   lo        lt         i       gt          hi
```

```cpp
int lt = lo, i = lo, gt = hi;
while (i <= gt) {
    if (a[i] < p) swap(a[lt++], a[i++]);
    else if (a[i] > p) swap(a[i], a[gt--]);     // do NOT advance i: a[i] is new
    else i++;
}
// recurse on [lo, lt-1] and [gt+1, hi]; the == block is done
```

Sort Colors (LC 75) is exactly this with the pivot 1: one pass, O(1) space. The bug to avoid: after swapping with `gt`, the element now at `i` has not been examined yet, so `i` must **not** advance.

5.5 Quickselect — the k-th smallest in O(n) average

After a partition, the pivot is at its final index p. If p is the index you want, done; otherwise **recurse into one side only**. Expected cost n + n/2 + n/4 + … = **O(n)**; worst case O(n²), made unlikely by a random pivot. `std::nth_element` is the library version. This is the answer to "can you beat O(n log n) for the k-th largest?" (LC 215).

PART 6 — Sorting without comparisons

6.1 The Ω(n log n) lower bound

Any sort that only compares pairs must, in the worst case, make **at least $\log_2(n!) \approx n \log_2 n$** comparisons: there are n! possible orderings, each comparison has two outcomes, so a decision tree that tells them all apart has height at least $\log_2(n!)$. Merge sort and heap sort are therefore **optimal** comparison sorts.

The only way to go faster is to **not compare** — to use the values themselves as indices.

6.2 Counting sort

When values lie in a small range 0 … k: count each value, then write them out in order.

```
vector<int> cnt(k + 1, 0);
for (int x : a) cnt[x]++;
int idx = 0;
for (int v = 0; v <= k; v++) while (cnt[v]--) a[idx++] = v;
```

O(n + k) time, O(k) space. Ideal for heights (LC 1051), grades, ages, characters, colours. The **stable** version — needed when sorting records by a key, and inside radix sort — converts counts to starting positions with a prefix sum and places records left to right.

6.3 Radix sort

Sort by the **least significant digit** first, then the next, …, using a **stable** counting sort per digit. After the last digit, the array is sorted, because each stable pass preserves the order established by the less significant digits. For d digits in base b: **O(d · (n + b))** — linear for fixed-width integers. Used for large arrays of 32-bit keys and for strings of equal length.

6.4 Bucket sort

Spread values uniformly into n buckets by range, sort each bucket (usually insertion sort), concatenate. O(n) **expected** for uniformly distributed input, $O(n^2)$ if everything lands in one bucket.

Maximum Gap (LC 164) uses buckets without sorting inside them: with n values spanning `max − min`, the largest gap is at least `(max − min) / (n − 1)`. Make buckets narrower than that, and the maximum gap can never be **inside** a bucket — only between one bucket's maximum and the next non-empty bucket's minimum. O(n).

	COUNTING	RADIX	BUCKET
Needs	small value range k	fixed number of digits d	uniform distribution
Time	O(n + k)	O(d · (n + base))	O(n) expected
Stable	yes (prefix-sum version)	yes	depends on the inner sort

PART 7 — Sorting in C++: the library and comparators

7.1 The functions

```
sort(a.begin(), a.end());                          // ascending, O(n log n), NOT stable
sort(a.begin(), a.end(), greater<int>());          // descending
sort(a.rbegin(), a.rend());                        // also descending
stable_sort(a.begin(), a.end(), cmp);              // keeps equal elements in order
partial_sort(a.begin(), a.begin() + k, a.end());   // only the k smallest, in order
nth_element(a.begin(), a.begin() + k, a.end());    // a[k] is the k-th; O(n) average
is_sorted(a.begin(), a.end());
reverse(a.begin(), a.end());
```

Sorting `vector<pair<…>>` or `vector<vector<int>>` with no comparator sorts **lexicographically**: by the first element, ties by the second. That is exactly right for intervals sorted by start.

7.2 Writing a comparator

`cmp(a, b)` returns `true` when **a must come before b**.

```
// by length, then alphabetically
sort(words.begin(), words.end(), [](const string& a, const string& b) {
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
});

// intervals by END (the greedy "activity selection" order)
sort(iv.begin(), iv.end(), [](const vector<int>& x, const vector<int>& y) { return x[1] < y[1]; });

// indices by the value they point to — keeps the original positions
vector<int> idx(n); iota(idx.begin(), idx.end(), 0);
sort(idx.begin(), idx.end(), [&](int i, int j) { return a[i] < a[j]; });
```

Take parameters by **const reference** — `vector<int>` by value copies on every comparison, and sorting does O(n log n) comparisons.

7.3 Strict weak ordering — the comparator rule that crashes programs

A comparator must behave like `<`, never like `<=`:

- `cmp(a, a)` must be **false**;
- if `cmp(a, b)` then not `cmp(b, a)`;
- it must be transitive.

```
sort(a.begin(), a.end(), [](int x, int y) { return x <= y; });   // UNDEFINED BEHAVIOUR
```

With `<=`, `std::sort` can read past the end of the array and **crash** — a real segmentation fault, typically only on large inputs with many equal values. This is one of the most confusing bugs in C++ because the comparator "looks right". Always use strict `<` or `>`.

7.4 A comparator that decides by concatenation (LC 179)

"Arrange numbers to form the largest number": put `a` before `b` when `a + b > b + a` as strings. `"9" + "34"` = `"934"` beats `"349"`, so 9 goes first. This ordering is transitive (a known proof), so `sort` is safe. Remember the all-zeros case: `[0, 0]` must give `"0"`, not `"00"`.

7.5 Stability, and when it matters

A stable sort keeps equal elements in their original order. That matters when you sort by **one key after another**: sort by the secondary key first, then **stable**-sort by the primary key, and ties on the primary key stay ordered by the secondary. (Or write one comparator that compares both keys.) `std::sort` is not stable; `std::stable_sort` is (merge-sort based, O(n log n), uses extra memory).

PART 8 — Sorting as the first step

8.1 Intervals

Sort by start, then sweep, keeping the current merged interval.

```
sort(iv.begin(), iv.end());
for (auto& cur : iv) {
    if (!res.empty() && cur[0] <= res.back()[1]) res.back()[1] = max(res.back()[1], cur[1]);
    else res.push_back(cur);
}
```

The `max` matters: `[1,4]` followed by `[2,3]` must stay `[1,4]`. The family: Merge Intervals (LC 56), Insert Interval (LC 57), Meeting Rooms (sort, check neighbours), Meeting Rooms II (sort starts and ends separately, or a min-heap of end times), Non-overlapping Intervals (LC 435 — sort by **end**, the activity-selection greedy, Topic 10).

8.2 Two pointers after sorting

Sorted input turns many pair problems into one linear pass: pair with a given sum (move the pointer on the side that is too big or too small), 3Sum (fix one, two-pointer the rest, skip duplicates), closest pair, squares of a sorted array (LC 977 — the largest square is at one of the two ends).

8.3 Sort to make a greedy correct

Many greedy choices are only valid on sorted input: assign cookies to children (sort both), boats to save people (sort, pair heaviest with lightest), minimum platforms at a station (sort arrivals and departures), H-index (sort, or count). Part of Topic 10.

8.4 Sort and keep the original index

When the answer must refer to original positions, sort `(value, index)` pairs or an index array (7.2). LC 1834 (Single-Threaded CPU), LC 506 (Relative Ranks) and LC 315 (Count Smaller After Self) all do this.

PART 9 — The complete pattern taxonomy

Every problem in Part 13 is labelled with one of these twelve. When you can name the pattern, you already have most of the code.

#	PATTERN	RECOGNISE IT BY	CORE TOOL
1	Exact binary search	"find x in a sorted array"	closed `[lo, hi]`, `lo <= hi`
2	Boundary search (lower / upper bound)	first / last occurrence, insert position, count	half-open `[lo, hi)`, first true
3	Monotone predicate	first bad version, sqrt, any yes/no that flips once	name the predicate, then Template 2 or 3
4	Peak / slope search	mountain, bitonic, "a peak"	compare `a[mid]` with `a[mid + 1]`
5	Rotated array search	"rotated sorted array"	one half is always sorted
6	2-D matrix search	sorted rows / columns	flatten the index, or the top-right staircase
7	Binary search on the answer	"minimum X such that", "minimise the max", "maximise the min"	bounds + greedy feasibility check
8	k-th by counting	k-th smallest in an implied set	binary search the value, count ≤ v
9	Partitioning	sort colours, k-th largest, segregate	Lomuto / Dutch flag / quickselect
10	Merge sort with counting	inversions, count smaller after self, range sums	count cross pairs while merging
11	Non-comparison sort	small value range, maximum gap	counting / radix / bucket
12	Sort, then scan	intervals, two pointers, greedy, custom order	`sort` + comparator, then one pass

How to tell 3 and 7 apart

Both binary search a predicate.

- **Pattern 3** searches over **positions** of something that already exists — an index, a version number.
- **Pattern 7** searches over **possible answers** that you have to verify by simulating — a speed, a capacity, a distance. The feasibility check is where the work is.

If you find yourself thinking "*if I knew the answer, could I check it quickly?*", you are in Pattern 7.

How to tell 9 and 10 apart

Both use the inside of a sorting algorithm.

- **Pattern 9** (partition) is about **where elements end up** — the k-th element, three groups.
- **Pattern 10** (merge) is about **counting pairs** with a cross-index condition — i < j and some relation between a[i] and a[j].

PART 10 — How to approach ANY searching or sorting problem

Step 1 — Read the constraints

$n \le 10^3$ allows $O(n^2)$ — an elementary approach may be fine. $n \le 10^5$ wants $O(n \log n)$ — sorting or binary search. A value range up to 10^9 with a cheap check suggests binary search on the answer (about 30 checks). Values in a small range suggest counting sort.

Step 2 — Is there something monotone?

Sorted input, a predicate that flips once, a feasibility question whose answer can only improve as the candidate grows. If yes, binary search — and **write the predicate down in words** before writing code.

Step 3 — Pick the template and state the invariant

Exact value → closed interval. First true → half-open, round down. Last true → round **up**. Say aloud what `lo` and `hi` mean.

Step 4 — For sorting problems: what is the key, and does stability matter?

Sort by what? Ties broken how? Do you need original indices (sort an index array)? Is the comparator strict?

Step 5 — Check the bounds and the types

Binary search bounds that include the answer (`hi = n` for "never true"; `hi = sum` for capacity). `long long` for products, sums of weights, `mid * mid`. `lo + (hi - lo) / 2`.

Step 6 — Dry-run the edge cases

1. **One element**, and an **empty** array if allowed.
2. The answer at **index 0** and at the **last index**.
3. A target **smaller than everything** and **larger than everything**.
4. **All elements equal** — duplicates break naive rotated search and two-way quicksort.
5. A range of exactly **two** elements — this is where `lo = mid` loops forever.

PART 11 — The bug catalogue

1. `(lo + hi) / 2` **overflows.** Symptom: wrong answers or an infinite loop only when indices or values are near 2^{31}. Use `lo + (hi - lo) / 2`.

2. Mixing the closed and half-open conventions. `hi = n` with `while (lo <= hi)` reads `a[n]`; `hi = mid` with `lo <= hi` can loop forever. Pick one convention per loop.

3. `lo = mid` **without rounding mid up.** Symptom: an infinite loop when `hi == lo + 1`. If a branch assigns `lo = mid`, use `mid = lo + (hi - lo + 1) / 2`.

4. Reading `a[lo]` **after the loop when** `lo` **can be** `n`**.** Symptom: out-of-bounds read when the target is larger than everything. Check `lo < n` first.

5. Wrong bounds in binary search on the answer. For ship capacity, `lo` must be the **heaviest single package**, not 1 — otherwise the feasibility check reports a capacity that cannot carry one item as feasible. Symptom: answers that are too small.

6. Overflow inside the feasibility check. `mid * mid` for sqrt, the sum of hours for Koko with piles of 10^9, the total for bounded arrays. Use `long long`.

7. Rotated search with duplicates. `a[lo] <= a[mid]` no longer proves the left half is sorted when `a[lo] == a[mid] == a[hi]`. Symptom: LC 81 / LC 154 fail on inputs like `[1,0,1,1,1]`. Shrink the ends when they are equal.

8. A comparator using `<=`**.** Violates strict weak ordering; `std::sort` can read out of bounds and crash on large inputs. Always strict.

9. Comparator taking `vector<int>` **by value.** Correct and slow — a copy per comparison. Use `const vector<int>&`.

10. Quicksort with a fixed pivot on sorted input. $O(n^2)$ and deep recursion — TLE or stack overflow. Use a random pivot or `std::sort`.

11. Advancing `i` **after swapping with the right end in a three-way partition.** The element swapped in has not been examined. Symptom: a 2 left in the middle of Sort Colors.

12. Merge sort allocating a new buffer at every call. Correct, and slow enough to TLE. Allocate once and pass it down.

13. `<` **instead of** `<=` **in the merge.** The sort loses stability, and inversion counts treat equal values as inverted. Take from the left on ties.

14. Forgetting the `max` **when merging intervals.** `[1,4],[2,3]` becomes `[1,3]`. Use `max(back.end, cur.end)`.

15. Using `std::lower_bound` **on a** `set` **or** `map`**.** The free function is O(n) on tree iterators. Use the member `s.lower_bound(x)`.

16. Expecting `std::sort` **to be stable.** Equal elements may be reordered. Use `stable_sort`, or put the tie-breaker in the comparator.

PART 12 — Complexity cheat sheet

ALGORITHM	BEST	AVERAGE	WORST	EXTRA SPACE	STABLE
Linear search	O(1)	O(n)	O(n)	O(1)	–
Binary search	O(1)	O(log n)	O(log n)	O(1)	–
Bubble sort (early exit)	O(n)	$O(n^2)$	$O(n^2)$	O(1)	yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	**no**
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	yes
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	yes
Quick sort	O(n log n)	O(n log n)	**$O(n^2)$**	O(log n) stack	no
Heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	no
Counting sort	O(n + k)	O(n + k)	O(n + k)	O(k)	yes
Radix sort	O(d(n + b))	O(d(n + b))	O(d(n + b))	O(n + b)	yes
Bucket sort	O(n)	O(n)	$O(n^2)$	O(n)	depends
`std::sort` (introsort)	O(n log n)	O(n log n)	O(n log n)	O(log n)	no
`std::stable_sort`	O(n log n)	O(n log n)	O(n log n)	O(n)	yes
Quickselect / `nth_element`	O(n)	O(n)	$O(n^2)$	O(1)	–
Binary search on the answer		O(n · log(range))		O(1)	–
Count inversions (merge)		O(n log n)		O(n)	–

The sentences to say in interviews:

> *"Each comparison halves the search space, so binary search takes $\log_2 n$ steps — about 30 for a billion elements."*
>
> *"If X works then every larger X works too, so feasibility is monotone in X and I can binary search the answer; each check is a greedy O(n) pass, giving O(n log range)."*
>
> *"Any comparison sort needs $\Omega(n \log n)$ comparisons in the worst case, because it must distinguish n! orderings; counting sort beats that only because it does not compare."*

PART 13 — The problem list

Work these in order. Every one is solved in the companion **Searching & Sorting — Solved Questions** PDF, in exactly this order — but attempt each problem yourself first and only read the solution after you have genuinely struggled with it.

Tier 1 — Learn the mechanics (do all thirteen first)

#	PROBLEM	PATTERN
704	Binary Search	1 — exact search ⭐
35	Search Insert Position	2 — lower bound
278	First Bad Version	3 — first true, overflow-safe mid
69	Sqrt(x)	3 — last true, round up
34	Find First and Last Position of Element in Sorted Array	2 — lower and upper bound ⭐
367	Valid Perfect Square	1 — exact, `long long`
744	Find Smallest Letter Greater Than Target	2 — upper bound with wrap
852	Peak Index in a Mountain Array	4 — slope
88	Merge Sorted Array	12 — merge from the back
912	Sort an Array	9 — randomised three-way quicksort ⭐
75	Sort Colors	9 — Dutch national flag ⭐
1051	Height Checker	11 — counting sort
977	Squares of a Sorted Array	12 — two pointers from the ends

Thirteen problems. When you can write 704, 34 and 75 from a blank page — and say what `lo` and `hi` mean at every step — you have the mechanics.

Tier 2 — The placement core (these are what actually get asked)

#	PROBLEM	PATTERN
33	Search in Rotated Sorted Array	5 — one half is sorted ⭐
153	Find Minimum in Rotated Sorted Array	5 — compare with the right end ⭐
162	Find Peak Element	4 — walk uphill
74	Search a 2D Matrix	6 — flatten the index
240	Search a 2D Matrix II	6 — top-right staircase
875	Koko Eating Bananas	7 — answer: speed ⭐
1011	Capacity To Ship Packages Within D Days	7 — answer: capacity ⭐
540	Single Element in a Sorted Array	3 — pair parity
981	Time Based Key-Value Store	2 — upper bound on timestamps
56	Merge Intervals	12 — sort by start ⭐
179	Largest Number	12 — concatenation comparator
215	Kth Largest Element in an Array	9 — quickselect
1482	Minimum Number of Days to Make m Bouquets	7 — answer: day
274	H-Index	11 — counting sort
81	Search in Rotated Sorted Array II	5 — duplicates

⭐ **Search in Rotated Sorted Array, Find Minimum in Rotated Sorted Array, First and Last Position, Koko Eating Bananas, Ship Packages, Merge Intervals and Sort Colors are the most-asked searching and sorting questions in on-campus drives** — Amazon, Microsoft, Adobe, Flipkart, Walmart, Goldman Sachs, TCS Digital, Infosys. Be able to write all of them cold.

Tier 3 — Harder, but they do appear

#	PROBLEM	PATTERN
4	Median of Two Sorted Arrays	7 — binary search the split
410	Split Array Largest Sum	7 — minimise the maximum (book allocation)
154	Find Minimum in Rotated Sorted Array II	5 — duplicates, O(n) worst
315	Count of Smaller Numbers After Self	10 — merge sort on indices
327	Count of Range Sum	10 — merge sort on prefix sums
719	Find K-th Smallest Pair Distance	8 — count pairs ≤ d
668	Kth Smallest Number in Multiplication Table	8 — count by formula
164	Maximum Gap	11 — buckets / pigeonhole
1095	Find in Mountain Array	4 — peak, then two searches
1802	Maximum Value at a Given Index in a Bounded Array	7 — maximise, arithmetic sums

The classic Indian-placement questions (GfG / written-round favourites)

These appear in written rounds and technical interviews, usually in GfG's wording:

- **Write bubble, selection and insertion sort**; trace them on a given array; count swaps and passes
- **Which sorts are stable? Which are in place? Best / worst cases?** — the Part 12 table
- **Write merge sort and quick sort**; explain the quicksort worst case and how to avoid it
- **Count inversions** in an array
- **Binary search — iterative and recursive**
- **First and last occurrence**; **count occurrences** in a sorted array
- **Floor and ceil** of x in a sorted array
- Find the **square root** of a number using binary search
- **Search in a rotated sorted array**; find the **rotation count** (index of the minimum)
- Find a **peak element**
- **Allocate minimum number of pages** (book allocation) — LC 410
- **Aggressive cows** — maximise the minimum distance
- **Painter's partition** — LC 410 again
- **Median of two sorted arrays**
- **Sort an array of 0s, 1s and 2s** — Dutch national flag
- **K-th smallest element** — quickselect or a heap
- **Merge two sorted arrays without extra space** (the gap method)
- Union and intersection of two sorted arrays
- Minimum number of **platforms** at a railway station

- **Find the missing and the repeating number**
- Sort a **nearly sorted** (k-sorted) array — insertion sort or a heap
- Minimum swaps to sort an array (cycle decomposition)

> **"Allocate minimum pages"** and **"aggressive cows"** are close to guaranteed in some form in Indian placements. Both are binary search on the answer: state the answer, the bounds, the greedy feasibility check, and whether you want the first true (minimise) or the last true (maximise).

PART 14 — A three-week plan

Assumes about 1.5 hours a day.

Week 1 — Binary search until it is automatic. Days 1–2: Parts 0–1. Write Templates 1, 2 and 3 from memory on paper, and for each one say what `lo` and `hi` mean. Days 3–5: Tier 1 binary searches — 704, 35, 278, 69, 34, 367, 744, 852 — and practice TODOs 8–9. Days 6–7: Part 2 — 33, 153, 162, 74, 240, 540, 981.

Week 2 — Binary search on the answer, and the sorts. Days 1–3: Part 2.5 — 875, 1011, 1482, 410, then aggressive cows (practice TODO 10) and book allocation. For every one, write the four-line recipe (answer, bounds, check, direction) before coding. Days 4–7: Parts 3–5 — practice TODOs 1–6 (bubble, selection, insertion, merge, count inversions, quick) **by hand**, then 88, 912, 75, 215.

Week 3 — Counting, comparators and hard. Part 6 and TODO 7, then 1051, 274, 164. Part 7–8: 56, 179, 977. Then Tier 3 as far as you get — 4 and 315 first, then 154, 327, 719, 668, 1095, 1802. Finish by re-solving **33, 34, 875, 1011, 56 and 75** on a timer — 15 minutes each, on paper, no compiler.

Ongoing: every time a problem defeats you, log which of the 12 patterns it was and which of the bugs you hit. After thirty problems that log tells you exactly what to drill.

Where this topic leads: heap sort and the k-th element return in Topic 07 (Heaps). Sorting first is the backbone of Topic 10 (Greedy). Binary search on the answer pairs with DP in Topic 11 (Longest Increasing Subsequence in O(n log n) is a `lower_bound` on a tails array). Merge sort with counting generalises to Fenwick and segment trees. Nothing here is wasted.

Quick reference card

```cpp
// ===== exact value: closed [lo, hi] =====
int lo = 0, hi = n - 1;
while (lo <= hi) { int mid = lo + (hi - lo) / 2;
    if (a[mid] == x) return mid;
    if (a[mid] < x) lo = mid + 1; else hi = mid - 1; }

// ===== FIRST true: half-open [lo, hi), round DOWN =====
int lo = 0, hi = n;
while (lo < hi) { int mid = lo + (hi - lo) / 2;
    if (P(mid)) hi = mid; else lo = mid + 1; }          // lo = first true (or n)

// ===== LAST true: round UP =====
while (lo < hi) { int mid = lo + (hi - lo + 1) / 2;
    if (P(mid)) lo = mid; else hi = mid - 1; }          // lo = last true

// ===== STL =====
lower_bound(a.begin(), a.end(), x) - a.begin();          // first >= x
upper_bound(a.begin(), a.end(), x) - a.begin();          // first >  x
s.lower_bound(x);                                        // on set / map: the MEMBER
sort(v.begin(), v.end(), [](const T& a, const T& b) { return a.k < b.k; });   // STRICT <
stable_sort(...);  nth_element(a.begin(), a.begin() + k, a.end());

// ===== rotated sorted array =====
if (a[lo] <= a[mid]) { if (a[lo] <= t && t < a[mid]) hi = mid - 1; else lo = mid + 1; }
else                 { if (a[mid] < t && t <= a[hi]) lo = mid + 1; else hi = mid - 1; }

// ===== binary search on the answer =====
lo = smallest possible answer; hi = largest possible answer;
while (lo < hi) { mid = lo + (hi - lo) / 2;
    if (feasible(mid)) hi = mid; else lo = mid + 1; }    // minimise; mirror to maximise

// ===== insertion sort =====
for (int i = 1; i < n; i++) { int key = a[i], j = i - 1;
    while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; } a[j + 1] = key; }

// ===== merge with inversion count =====
if (a[i] <= a[j]) tmp[k++] = a[i++];
else { inv += mid - i + 1; tmp[k++] = a[j++]; }

// ===== Lomuto partition =====
int p = a[hi], i = lo;
for (int j = lo; j < hi; j++) if (a[j] < p) swap(a[i++], a[j]);
swap(a[i], a[hi]);                                       // pivot final at i

// ===== Dutch national flag =====
while (i <= gt) { if (a[i] < p) swap(a[lt++], a[i++]);
                  else if (a[i] > p) swap(a[i], a[gt--]); else i++; }

// ===== counting sort =====
for (int x : a) cnt[x]++;  for (v = 0..k) while (cnt[v]--) a[idx++] = v;
```

The five rules, one last time:

1. **Binary search needs a monotone predicate, not a sorted array.** Name the predicate first.
2. **Say what `lo` and `hi` mean** — closed or half-open — and make every update keep that meaning.
3. **`lo = mid` → round mid up.** `mid = lo + (hi − lo) / 2` always.
4. **"Minimise the maximum" / "maximise the minimum" → binary search on the answer** with a greedy check.

5. **Comparators are strict (`<`, never `<=`)**, and `std::sort` is not stable.